Milestone-3 Documentation

Exoplanet Habitability Prediction System

Module 5 & Module 6

MODULE 5: Backend Theory

1. Introduction

The backend exposes the trained Machine Learning model as a REST API using Flask. It accepts planetary and stellar parameters as input and returns habitability predictions along with probability scores.

2. Backend Environment Setup

The backend environment was configured using Python. The required libraries include:
- Flask
- joblib
- numpy
- pandas

The requirements.txt file ensures reproducibility of the environment.

3. Backend Folder Structure

```
backend/
├── app.py
├── utils.py
└── models/
    └── final_model.pkl
```

4. Model Loading

The trained ML model is saved as a .pkl file using joblib. The model loads automatically when the Flask application starts, ensuring optimized performance and fast predictions.

5. API Design

1. /predict (POST)
   - Accepts exoplanet parameters in JSON format
   - Returns habitability prediction and confidence score

2. /rank (GET/POST)
   - Returns ranked list of exoplanets based on habitability score

6. Input Validation

The backend validates incoming JSON data to ensure:
- No missing parameters
- Correct numeric data types
- Valid parameter ranges

Meaningful error messages are returned in case of invalid input.

7. Response Structure

All responses follow a structured JSON format containing:
- status
- prediction
- habitability score
- message (if error)

8. Backend Testing

APIs were tested using Browser, Postman, and curl. Testing ensured correct predictions, proper formatting, and stable server performance.

9. Backend Theory

The backend follows client-server architecture:
User → Frontend → Backend API → ML Model → Prediction → Response

The ML model processes numerical inputs and generates habitability classification with associated probability.

MODULE 6: Frontend UI Development

1. Introduction

The frontend provides a user-friendly interface that allows users to input exoplanet data and view habitability predictions. It is built using HTML, CSS, Bootstrap, and JavaScript.

2. Frontend Folder Structure

```
frontend/
├── index2.html
├── style2.css
└── script2.js
```

3. UI Layout Design

The layout includes input forms, submit button, and result display section. Bootstrap ensures a clean, responsive design across desktop and mobile devices.

4. Input Form Creation

Input fields include planetary and stellar parameters such as:
- Planet radius
- Planet mass
- Orbital period
- Stellar temperature

Client-side validation ensures proper input before submission.

5. Connecting Frontend to Backend

JavaScript Fetch API sends user input as JSON to the /predict endpoint. The response is handled asynchronously without reloading the page.

6. Display Prediction Results

Results display:
- Habitability status
- Confidence score

Displayed using Bootstrap cards, alerts, or tables.

7. Error Handling

User-friendly error messages are shown for invalid inputs or backend failures. Page reload is prevented to ensure smooth user experience.

8. Integration Testing

Complete system flow tested:

User Input → Frontend → Backend → ML Model → Response → UI

Verified correctness, performance, and stability.

9. Frontend Theory

The frontend follows client-side rendering principles. It communicates with the backend via HTTP requests and dynamically updates the webpage using DOM manipulation.

Conclusion

Modules 5 and 6 successfully integrate Machine Learning, REST API, and an interactive Web Interface to build a complete end-to-end Exoplanet Habitability Prediction System.